

Méthodes d'adaptation d'un LLM pour la classification de sentiments

Objectif : Implémenter et comparer trois méthodes d'adaptation (Fine-Tuning complet, LoRA, Distillation) pour une tâche de classification de sentiments sur Distil-GPT2/Pythia-160m.

Question 1 - Configuration de l'environnement

Installez les bibliothèques nécessaires (``transformers``, ``datasets``, ``accelerate``, ``evaluate``, ``peft``) et importez celles déjà disponibles (``pandas``, ``matplotlib``, etc.) dans Colab. Configurez l'environnement pour assurer la reproductibilité (`seed=42`) et vérifiez la disponibilité d'un GPU.

Question 2 - Génération du dataset

Créez une fonction ``generer_dataset_sentiments()`` qui retourne un DataFrame pandas contenant :

- 20 avis positifs, 20 négatifs et 20 neutres
- Des labels : 0=négatif, 1=neutre, 2=positif
- Une augmentation simple des données (variations de ponctuation et casse) pour tripler le dataset

Question 3 - Split des données

Divisez le dataset en ensembles d'entraînement (80%) et de test (20%) en préservant la distribution des classes.

Affichez les tailles et distributions.

Question 4 - Chargement du modèle baseline

Chargez le tokenizer et le modèle relatifs à ``distilgpt2`` depuis Hugging Face. Adaptez le modèle pour la classification de 3 classes.

Question 5 - Fine-tuning

Chargez un nouveau modèle `model_finetuned` à tokenizer au préalable. Définissez les `TrainingArguments` (3 époques, batch 8, etc.). Instanciez le `Trainer` et lancez `trainer.train()`.

Question 6 - LoRA (PEFT)

Chargez un nouveau modèle `model_lora`. Définissez une `LoraConfig` (`r=8`, etc.) et appliquez la configuration avec `get_peft_model`. Comparez les paramètres entraînaables avec le modèle complet. Instanciez un `Trainer` pour ce modèle LoRA et lancez l'entraînement.

Question 7 - Distillation

Définissez le `model_baseline` comme `teacher_model` et créez un `student_model` plus petit via `AutoConfig` personnalisé. Créez un `DistillationTrainer` puis instanciez et lancez l'entraînement de l'élève.

Question 8 - Évaluation

Créez une fonction `evaluer_modele` (métriques + matrice de confusion). Évaluez les 4 modèles sur le jeu de test : baseline, fine-tuné, LoRA, et distillé.
Stockez tous les résultats pour l'analyse.

Question 9 - Visualisation

Visualisez l'Accuracy globale ainsi que les Recall et Precision globaux et par classe, puis les matrices de confusion des 4 modèles.

Illustrez l'ensemble des prédictions puis comparez également l'efficacité (nombre de paramètres et temps d'exécution soit automatiquement, soit en transcrivant manuellement le nombre de minutes en secondes de la fenêtre).

Question 10 - Conclusion

Justifiez selon vous quel est le meilleur compromis performance/efficacité.